

Software and Applications

Lesson Objectives

By the end of this lesson, students should be able to:

- Differentiate between **system software** and **application software**.
 - Describe the **evolution of programming languages**.
 - Explain **software installation and maintenance** processes.
 - Install a text editor or IDE and manage notes using **GitHub**.
-

1 Introduction to Software

A **computer system** is made up of **hardware** (the physical components) and **software** (the set of instructions that tell the hardware what to do).

Definition:

Software is a collection of data, programs, and instructions that enable a computer to perform specific tasks.

Analogy:

If hardware is the body of a computer, software is its brain.

2 Types of Software

1. System Software

System software is designed to **operate and control the computer hardware** and provide a platform for running application software.

Main Categories:

- **Operating Systems** (Windows, Linux, macOS)
- **Device Drivers** (Printer, Keyboard, GPU drivers)
- **Utility Programs** (Antivirus, Disk Cleanup, Backup tools)

Examples:

System Software	Function
Operating System	Manages resources and provides user interface
Device Drivers	Enable hardware to communicate with OS
Utility Programs	Maintain and optimize system performance

Key Features:

- Runs automatically when the computer starts.
- Bridges hardware and user applications.
- Provides essential services for application software.

2. Application Software

Application software performs **specific user-oriented tasks**.

◆ Examples by Category:

Category	Examples	Purpose
Word Processing	MS Word, Google Docs	Create text documents
Spreadsheets	Excel, Google Sheets	Data analysis and management
Graphics	Photoshop, Canva	Image editing and design
Database	MySQL, Access	Store and manage data
Web Browsers	Chrome, Firefox	Access online resources
Education	MATLAB, Scratch	Learning and teaching tools

Key Features:

- User-specific and task-oriented.
- Installed on top of the operating system.

- Can be proprietary or open-source.

3 Difference Between System and Application Software

Feature	System Software	Application Software
Purpose	Manages system operations	Performs user tasks
Runs On	Hardware	System Software
Examples	Windows, Linux, BIOS	MS Word, Photoshop
Installation	Pre-installed or system setup	Installed by user
Dependency	Essential for system	Optional
Users	System administrators	End users

4 Programming Languages Overview

Programming languages are used to **create software**.

They have evolved through generations to become more user-friendly and powerful.

Evolution Stages:

Generation	Language Type	Description	Example
1st Gen (Machine Language)	Binary code	Written in 0s and 1s	10110010
2nd Gen (Assembly Language)	Uses mnemonics	Symbolic representation of machine code	MOV A, 5
3rd Gen (High-Level Language)	Human-readable syntax	Easier to understand and maintain	Python, C++, Java
4th Gen (Declarative Languages)	Focus on <i>what</i> to do	Used in database and automation	SQL
5th Gen (AI-Oriented)	Uses logic and AI	Natural language and inference-based	Prolog, LISP

Example Comparison:

Language Type	Code to Add Two Numbers
Machine Language	10110000 01100001
Assembly Language	MOV AL, 1; ADD AL, 2
High-Level Language	sum = 1 + 2

Observation:

As languages evolved, they became easier to read, write, and maintain, reducing errors and improving productivity.

Software Installation and Maintenance

Software Installation Steps

1. **Download or obtain the installer.**
2. **Run the installer program.**
3. **Follow setup instructions** (select location, options).
4. **Verify installation** (check shortcuts or menus).
5. **Activate or register** (if required).

Common Installation Types:

- **Full Installation:** Installs all features.
 - **Custom Installation:** User selects components.
 - **Network Installation:** Installs software from a server.
-

Software Maintenance

Software requires regular **updates and checks** to stay secure and efficient.

Types of Maintenance:

Type	Purpose
Corrective	Fixes bugs and errors.
Adaptive	Adapts software to new environments.
Perfective	Enhances features or performance.
Preventive	Prevents future issues through optimization.

Example:

Regularly updating your antivirus ensures protection against new threats.

Class Activity

Objective:

Identify examples of system and application software on your computer.

Instructions:

1. Open your computer and list:
 - o 3 system software examples.
 - o 5 application software examples.
2. For each, include:
 - o Name
 - o Function
 - o Installation type (pre-installed or user-installed)

Sample Table:

Software Name	Type	Function	Installation
Windows 10	System	Manages computer hardware	Pre-installed
Chrome	Application	Browsing web	User-installed
MS Word	Application	Word processing	User-installed



Lab: Install a Text Editor or IDE and Use GitHub

Objective:

Install and configure a text editor or IDE, then use Git to track notes.

Instructions:

1. **Choose a text editor or IDE:**

- VS Code, Sublime Text, Atom, PyCharm, or Notepad++.

2. **Install your editor:**

- Download from the official site.
- Follow installation prompts.

3. **Initialize a Git repository:**

4. `mkdir software_lab`
5. `cd software_lab`
6. `git init`

7. **Create a markdown file:**

8. `echo "# My First IDE Notes" > ide_notes.md`

9. **Write a short note:**

Include:

- The name of your editor.
- Why you chose it.
- One feature you like most.

10. **Commit and push:**

11. `git add ide_notes.md`
12. `git commit -m "Added IDE installation notes"`
13. `git remote add origin https://github.com/yourusername/software_lab.git`
14. `git push -u origin main`